Anthropic 官方文档 · 中文精读

你打不开的那个站，内容在这里 —— 附实测可用的替代链接

整理日期：2026-10-06 ｜ 素材来源：Anthropic 官方文档纯文本接口（`llms-full.txt`，41MB）+ 官方博客原文
本文是**官方原文的结构化中文整理**，不是逐句翻译；涉及具体参数处保留英文原词，便于你交叉核对。

一、先给结论：不是链接失效，是地区限制

你截图里那句 "App unavailable in region" 就是答案。
链接本身是好的，是 **Anthropic / Claude 全系域名不对中国大陆提供服务**。这不只影响那一个页面 —— `docs.anthropic.com`、`docs.claude.com`、`platform.claude.com`、`www.anthropic.com` 全部一样。所以别去换链接了，换域名才是正事。

我这边的实测结果（重要前提）
我这边能打开 Anthropic 文档（HTTP 200），你那边不行 —— 说明两边出口地区不同。因此下表状态**仅供参考，请以你本机实际打开为准**。我连不上 GitHub 和 HuggingFace，但这不代表你也连不上。

资源	我这边实测	建议
Anthropic 官方文档 / 博客	**你那边被区域拦截**	内容已整理成本文，见第二、三节
Prompt Engineering Guide 中文版	**200 可访问**	**首选替代。**DAIR.AI 出品，覆盖提示词工程全貌，中文，质量高，和 Anthropic 文档内容高度重叠
通往 AGI 之路（WayToAGI）	**200 可访问**	中文 AI 知识库，查工具选型、中文实践案例好用
DeepLearning.AI 短课程	**200 可访问**	吴恩达团队，W1/W4 的指定课程，能开就能用
Google 提示词工程白皮书（Kaggle）	**200 可访问**	W1 作业要用，免费下载 PDF

GitHub（`anthropics/courses` 等）	我这边连不上	❓ 请你自己试一下。国内一般能开网页版，只是速度不稳。若能开，Anthropic 官方课程仓库是极好的免费资源
HuggingFace	我这边连不上	❓ 同上，需要你本机验证

> **一个可以长期用的办法：**Anthropic 文档站提供纯文本接口 `llms-full.txt`，我能直接抓下来（这次抓到了 41MB，含 753 篇文档）。**以后你想看 Anthropic 官方任何一篇文档，跟我说一声，我抓下来给你整理成中文**，不用管那个域名能不能开。

二、提示词工程官方指南 · 中文精读

对应原文：Prompting best practices（官方称其为「living reference」，即持续更新的活文档）

2.1 动手之前，先确认三件事

官方开篇就把话说死了 —— 这份指南**假设你已经具备**：

1. 对使用场景有**明确的成功标准**
2. 有**能实测这些标准的方法**
3. 有一版**想改进的初稿 prompt**

如果还没有，先去建立这些。官方原话指向「Define success criteria and build evaluations」。

> **一个反直觉但极重要的提醒：不是所有问题都该用 prompt 解决。**
> 官方原文明确指出：有些失败案例，**换个模型比改 prompt 更省事、更便宜**（原话大意：sometimes improve latency and cost more easily by selecting a different model）。这句话值得贴在显示器上 —— 「效果不好就加提示词」是新手最容易陷入的泥潭。

2.2 五条通用原则

① 清晰、直接（Be clear and direct）

把模型想象成一个**极聪明但对你的规矩一无所知的新员工**。你说得越精确，结果越好。

> **黄金法则：**把你的 prompt 拿给一个完全不了解这项任务的同事，让他照着做。如果他看不懂，模型也看不懂。

两个具体动作：

- 把**输出格式和约束**说清楚
- 如果步骤顺序重要，用**编号或项目符号**列出，别写在长句里

官方例子对比 —— 差：「做一个数据分析看板」 / 好：「做一个数据分析看板。包含尽可能多的相关功能和交互。不要停留在基础版，做成完整功能的实现。」

② 说明动机，而不只是下命令

解释**为什么**要这么做，模型能理解意图并推广到相似情况。

官方例子 —— 差：「绝对不要用省略号」 / 好：「这段文字会被语音合成引擎朗读，所以绝不要用省略号，因为语音引擎不知道怎么读它。」

同样的道理：与其说「简短点」，不如说「阅读者是没时间的技术负责人，200 字内让我能判断该不该继续读」。

③ 用例子（few-shot）—— 最可靠的手段

官方称例子是**控制输出格式、语气、结构最可靠的方法。**要求三条：

- **相关**：贴近你的真实用例
- **多样**：覆盖边界情况，否则模型会学到你没想要的模式
- **结构化**：包在 `<example>` 标签里（多个用 `<examples>` ），让模型能区分例子和指令

官方给的数字：**3-5 个例子效果最好。**还可以让模型自己评估你的例子够不够相关、够不够多样，或基于初稿补充例子。

④ 用 XML 标签给提示词分块

当 prompt 里同时混着**指令、背景、例子、待处理的输入**时，XML 标签能让模型毫无歧义地解析。给每类内容单独包一个标签，比如 `<instructions>` 、 `<context>` 、 `<input>` 。

- 标签名保持**一致且描述性**，别每次换名字

- 内容有天然层级时用**嵌套**（文档放 `<documents>` 里，每篇再包 `<document index="n">`）

⑤ 给模型一个角色

在系统提示词里设定角色，能聚焦行为和语气。**哪怕只有一句话也有效果。**

例如 system：`You are a helpful coding assistant specializing in Python.`

2.3 长上下文技巧（做 RAG 必看）

处理 20k+ token 的长文档时，结构比内容更重要：

- **长数据放最前面**：长文档、待处理输入放在查询和指令**之前**，所有模型都受益
- **查询放最后**：官方给的数据 —— 查询放末尾能**提升最多 30%** 的回答质量，多文档复杂输入时尤其明显
- **多文档用 XML 分层**：每篇包 `<document>`，内含 `<document_content>` 和 `<source>` 元数据
- **先引用、再作答**：要求模型先摘出文档里相关的原文片段（放进 `<quotes>` 标签），再基于引文回答。这能帮模型聚焦相关内容、忽略无关部分

> **写给你的话：**最后一条「先引用、再作答」是做 RAG 时的救命技巧。知识库里检索出 20 个片段，模型经常抓不住重点 —— 让它先引用原文、再基于引文回答，效果会有明显变化。**已排入学习计划的 W4 任务**，和「召回率/准确率」那两个指标是一对。

2.4 控制输出格式

1. **说「要做什么」，别说「不要做什么」**
   差：「不要在回答里用 markdown」　好：「你的回答应该由流畅的散文段落组成」
2. **用 XML 格式指示器**
   例如：「把回答的散文部分写在 <smoothly_flowing_prose_paragraphs> 标签里」
3. **让 prompt 的风格与目标输出一致**
   官方观察：prompt 里的排版风格会影响输出的排版风格。想减少输出里的 markdown，先从自己的 prompt 里去掉 markdown

4. **要精确控制就写详细指令**
   官方给了一整段可复用的指令块（大意）：写报告、文档、技术说明这类长文时，用完整的段落和句子，保留 markdown 只在行内代码、代码块和简单标题上；除非确实是离散条目或用户明确要求列表，否则不要用有序/无序列表；把条目自然融进句子里；**绝不输出一连串过短的要点。**目标是让读者顺畅地跟着思路走，而不是把信息碎成一地孤点。

顺带一条：官方提到新版模型默认用 LaTeX 写数学表达式。要纯文本的话，得明确写指令要求（用 / * ^ 这类标准字符）。

2.5 工具使用：怎么让 AI 真的动手

核心问题：如果你说「你能建议一些改进吗」，模型可能只给建议而不动手 —— 哪怕你其实想让它直接改。

差：「你能建议一些改进吗？」（只给建议）　好：「修改这个函数以提升性能。」（直接动手）

两种情况可以分别用这两段可复用的系统提示词：

```
<default_to_action>
默认直接实施改动，而不只是提建议。如果用户意图不明确，
推断最可能的有用动作并执行，用工具去发现缺失的细节而不是猜测。
</default_to_action>
```

```
<do_not_act_before_instructions>
除非明确被要求改动，否则不要进入实施阶段、不要改文件。
用户意图模糊时，默认提供信息、做调研、给建议，而不是动手。
</do_not_act_before_instructions>
```

另有一条实用提醒：新版模型对系统提示词更敏感，如果以前的 prompt 为了「确保它调用工具」写了大量强调（比如「CRITICAL: You MUST use this tool」），现在会**过度触发。**修法是降低语气强度，改成「Use this tool when...」。

2.6 思考与推理

新版模型的思考（thinking）是**自适应**的 —— 模型自己判断什么时候想、想多久，由 `effort` 参数和问题复杂度共同决定。复杂问题想得多，简单问题直接回答。

两种常见调节需求：

- **嫌它想太多、太慢**：可以给指令约束，比如「决定方案后就执行，不要反复推翻，除非有新信息直接与你的推理矛盾。在两个方案间权衡时，选一个做到底，失败了再纠正」；或直接调低 `effort`
- **想让它更主动规划**：可以要求「收到工具结果后，先反思结果质量、确定下一步，再用思考来规划和迭代」

2.7 Agentic 系统要点

长任务的状态管理

- **状态用结构化格式**：测试结果、任务状态这类信息用 JSON 存（例如 `tests.json`），模型更容易理解 schema 要求
- **进度用自由文本**：进度笔记（`progress.txt`）适合记录总体进展和上下文
- **用 git 跟踪状态**：git 天然提供「做过什么」的日志和可回滚的检查点，跨会话跟踪表现尤其好
- **强调增量推进**：明确要求模型聚焦「一次做一点、稳步前进」，而不是一次吞下所有事

自主与安全的平衡（做 Agent 必须设的护栏）

不加约束时，模型可能做出难以撤销的动作。官方建议明确区分：

- **鼓励**：本地、可逆的操作（改文件、跑测试）
- **必须先问人**：破坏性操作（删文件/分支、drop 表、`rm -rf`）、难以撤销的操作（`git push --force`、`git reset --hard`、改写已发布的提交）、对他人可见的操作（推代码、评论 PR、发消息、改共享基础设施）

还有一条：遇到障碍时，不许拿破坏性操作当捷径 —— 比如绕开安全检查（`--no-verify`）或丢弃不明来源的文件。

prompt 链式（self-correction 自我纠错）

最常见也最有用的模式：**生成初稿 → 让模型按标准评审 → 让模型根据评审改进**。每一步是一次独立调用，所以你可以随时记录、评估、分支。

这就是 W10 要做的「评测流水线」的雏形 —— 官方把它列为 Agentic 场景的主力模式。

> **关于模型名的一点说明：**官方文档里提到的 Claude Fable 5.1、Mythos 5.1、Opus 5.5、Sonnet 5.5 等，是 Anthropic 当前（2026 年）的模型世代划分。这些是**Claude 专属细节**，对我们用国产模型落地没有直接影响 —— 你要带走的是**通用原则**（清晰、给例子、XML 分层、先引用再作答、说做而不说别做），这些跨模型全部适用。

三、《Building Effective Agents》中文精读

对应原文：Building effective agents，Anthropic 工程博客，发布于 2024-12-19，作者 Erik S. 与 Barry Zhang。这是 Agent 领域必读的第一篇。

> **先说清一个时间问题（诚实标注）：**官方在文章顶部加了一条提示 —— **文中描述的工具体系自 2024 年 12 月以来已大幅变化**，并指向了新的 Managed Agents 文档。所以：**五种模式的设计思想仍然有效且经典，但里面提到的具体框架名不要当成 2026 年的推荐清单。**网上很多中文文章转的还是 2024 年的框架（LangChain/LangGraph），那已经是旧版本了。

3.1 最核心的区分：workflow ≠ agent

官方把两者统称为 **agentic systems（智能体系统）**，但强调一个关键的架构区别：

类型	定义	特征
Workflow 工作流	LLM 和工具通过**预先定义的代码路径**来编排	可预测、一致、好调试。适合定义清晰的任务
Agent 智能体	LLM **动态决定**自己的流程和工具使用，掌握「如何完成」的控制权	灵活、能处理开放式问题，但成本高、错误会累积

> **全文最值钱的一句话：**很多应用场景，**把单次 LLM 调用加上检索和少量示例优化好，就已经足够了**（optimizing single LLM calls with retrieval and in-context examples is usually enough）。

> **这句话能省你三个月：**新手最常犯的错是「上来就做自主 Agent」。官方反复强调：**先用最简单的方案，只在确有必要时才增加复杂度**，甚至可能根本不需要做 agentic 系统。复杂度换来的是延迟和成本，得先算这笔账值不值。

3.2 五种 workflow 模式

① Prompt chaining（提示词链）

把任务拆成一系列步骤，每一步处理上一步的输出，中间可以加程序化检查（gate）确认没跑偏。

什么时候用：任务能被干净地拆成固定的子任务。用**牺牲延迟换准确率** —— 每一步都更简单，模型更容易做对。

例子：先写营销文案再翻译成另一语言；先写大纲、检查大纲是否达标、再按大纲写正文。

② Routing（路由分流）

先分类输入，再导向专门的处理流程。好处是**关注点分离** —— 为某一类输入优化不会损害其他输入的表现。

什么时候用：任务有明确可分的类别，且分类本身能做好（模型或传统分类器都行）。

例子：客服问题分流到不同流程；**简单问题路由给便宜的小模型、难问题给强模型** —— 这条直接省钱。

③ Parallelization（并行化）

两种变体：

- **Sectioning 切分**：把任务拆成独立子任务并行跑
- **Voting 投票**：同一任务跑多次，取多样的输出

什么时候用：子任务能并行加速，或需要多个视角/多次尝试来提高置信度。官方观察：复杂任务里，**让每个考量点由一次独立的 LLM 调用处理，效果通常更好**（注意力更集中）。

例子：一个模型实例处理用户请求、另一个实例同时筛查不当内容（护栏），比让同一个调用兼做两件事效果更好；代码安全审查用多个不同 prompt 分别找漏洞。

④ Orchestrator-workers（编排者-工作者）

一个中心 LLM **动态拆解任务**，派给若干 worker LLM，再汇总结果。

什么时候用：子任务**事先无法预测**的复杂任务。（这和并行化的关键区别就是灵活性 —— 子任务不是预先定义的，而是编排者根据具体输入决定的。）

例子：需要改多个文件的编码任务（改几个文件、每个文件改什么，取决于任务本身）；需要从多个来源收集分析信息的搜索任务。

⑤ Evaluator-optimizer（评估者-优化者）

一个 LLM 生成回答，另一个 LLM 评估并给反馈，形成循环。

什么时候用：有**明确的评估标准**，且迭代改进能带来可衡量的价值。两个适配信号：人说出反馈时回答能明显变好；模型自己也能提供这种反馈。

例子：文学翻译（细微之处初稿捕捉不到，评估者能给出有用的批评）；需要多轮搜索分析才能收集完整信息的复杂检索任务。

3.3 什么时候才该用 agent

Agent 的典型形态其实很朴素：就是 LLM 在一个循环里，基于环境反馈使用工具。所以**工具集和文档的设计至关重要**。

适用条件：开放式问题，步骤数难以预测、无法硬编码固定路径，且你**对它的决策有一定程度的信任**。

代价（官方明确点出）：自主性意味着**更高的成本**和**错误累积**的可能。建议在沙箱环境里充分测试，并配上适当的护栏。

官方自己的例子：解决 SWE-bench 任务的编码 Agent；「computer use」参考实现。

3.4 三条核心原则（全文精华）

1. **保持简单**（Maintain simplicity in your agent's design）
2. **优先透明** —— 明确展示 agent 的规划步骤（Prioritize transparency by explicitly showing the agent's planning steps）

3. **精心打磨 ACI**（agent-computer interface，智能体-计算机接口）—— 通过完善的工具文档和测试把接口做好

> 官方总结：「LLM 领域的成功不在于构建最复杂的系统，而在于为你的需求构建合适的系统。从简单 prompt 开始，用充分的评估来优化，**只有当更简单的方案不够用时，才引入多步 agentic 系统。**」

3.5 工具设计（Appendix 2，实操性最强）

官方观点：**工具定义需要和整体 prompt 花同样的提示词工程心思。**操作方式可以有很多种（比如改文件可以写 diff、也可以整文件重写），但有些格式对 LLM 来说难得多 —— 写 diff 要求提前算好改了几行；把代码塞进 JSON 要多做转义。

选择工具格式的三条建议：

- 给模型**足够的 token 去「想」**，别让它把自己逼进死角
- 格式尽量**贴近模型在网上见过自然出现的形式**
- 确保没有格式上的**额外负担**（比如要精确数数千行代码，或给它写的每段代码做字符串转义）

> 一句话记住：**你在人机界面（HCI）上花多少功夫，就该在智能体-计算机接口（ACI）上花同样多功夫。**

具体做法：

- **站在模型的角度想**：光看描述和参数，这个工具怎么用是否一目了然？好的工具定义通常包含使用示例、边界情况、输入格式要求，以及和其他工具之间的清晰边界
- **改参数名和描述**让它更显然 —— 就像给你团队里的初级工程师写一份好的 docstring（工具多且相似时尤其重要）
- **实测模型怎么用你的工具**：跑大量示例输入，看它犯什么错，然后迭代
- **防呆设计（poka-yoke）**：改参数设计，让错误更难发生

官方实战案例：在做 SWE-bench agent 时，**他们花在优化工具上的时间比优化整体 prompt 还多。**具体发现 —— 当 agent 离开根目录后，模型会对使用相对路径的工具犯错；改成强制要求绝对路径后，模型用起来毫无问题。

四、给你的三条实操建议

1. 把 URL 换成能力，而不是换成另一个 URL。
与其满世界找能打开的镜像站，不如记住这件事：**Anthropic 官方内容我能抓（llms-full.txt）。**以后你在任何文章里看到「Anthropic 官方文档说……」，直接把链接或标题丢给我，我给你抓原文并整理。

2. 把 promptingguide.ai/zh 加入 W1 的书签。
它是这次唯一一个既全面、又中文、又能直接打开的替代品。W1 那周「通读官方 prompt 文档」这条任务，就改用它 + 本文第二节。

3. 第二节的「先引用、再作答」在 W4 就会用到。
这条属于**改一行 prompt 就能明显提升准确率**的少数技巧，已排进计划页的 W4 任务。它和「长数据放前面、查询放最后」是一组。你搭第一个知识库时直接试，对比一下加不加的差别 —— 这本身就是一个很好的评测实验，顺便为 W5 的评测集打个底。

来源与说明

· 提示词工程部分：整理自 Anthropic 官方文档 Prompting best practices / Prompt engineering overview（经 `docs.anthropic.com/llms-full.txt` 获取，2026-10-06 抓取，文件 40,988,195 字节，含 753 篇文档）

· Agent 部分：整理自 Anthropic 工程博客 Building effective agents（原文发布于 2024-12-19，抓取于 2026-10-06）

· 本文为**结构化中文整理**，非逐句翻译；关键术语与参数保留英文原词以便核对。凡属作者（Anthropic）原话之处已用引用样式标出。

· 表格中的可访问性状态为抓取时的实测结果，**不代表你的网络环境**，请以本机为准。

· 整理：巴蒂　2026-10-06